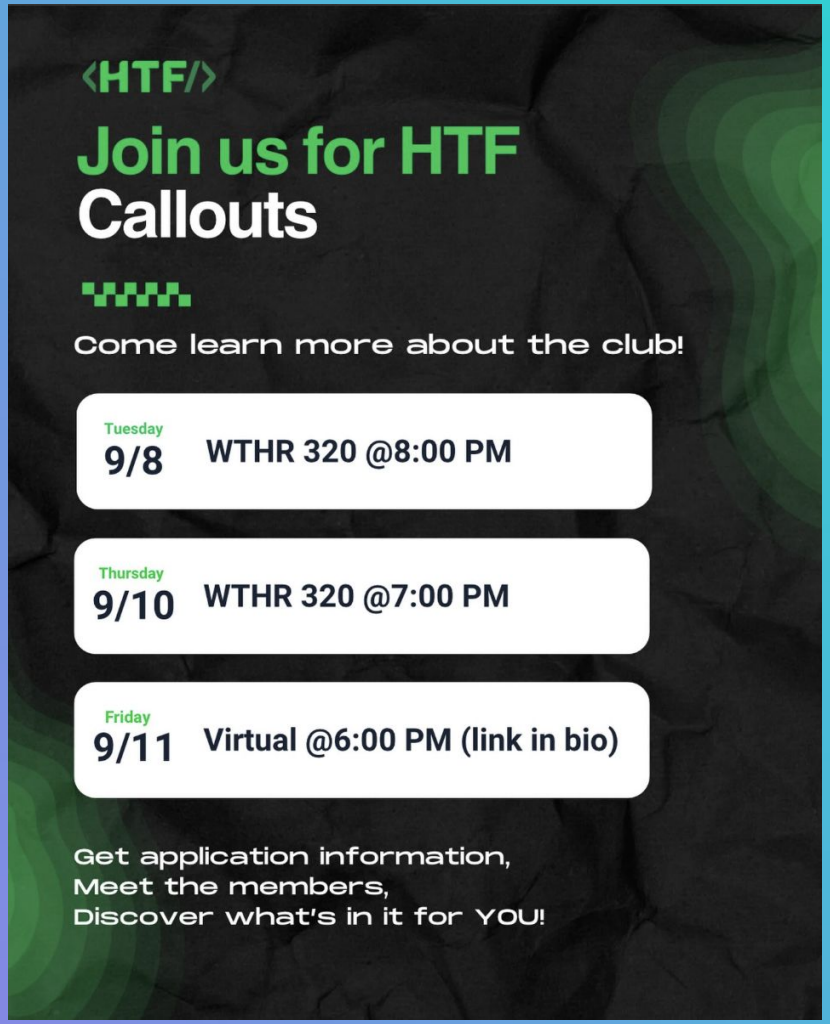
CS 193

Lecture 3: Bash Continued

LOGIN TO data via SSH while waiting (remember Homework 1?)

Help a neighbor while you're at it :)

Announcements!


⟨HTF/⟩
Join us for HTF
Callouts
Come learn more about the club!
Tuesday
9/8
WTHR 320 @8:00 PM
Thursday
9/10
WTHR 320 @7:00 PM
Friday
9/11
Virtual @6:00 PM (link in bio)
Get application information,
Meet the members,
Discover what's in it for YOU!

ML@PURDUE

project member applications

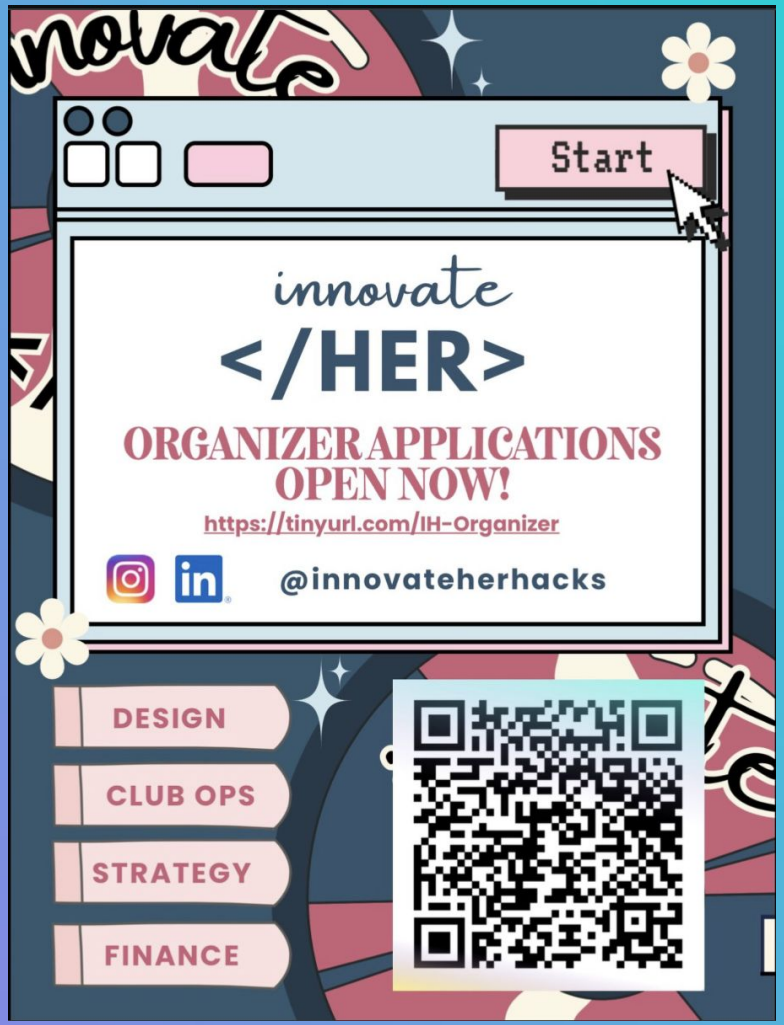
due date tbd

ML@PURDUE



accelerator mentee applications

due 9/11


Start
innovate
</HER>
ORGANIZER APPLICATIONS OPEN NOW!
https://tinyurl.com/IH-Organizer
@innovateherhacks
DESIGN
CLUB OPS
STRATEGY
FINANCE

Lecture 2 Review

- Discussed how to find and join different student organizations using BoilerLink and campus club fairs. Had club ambassadors come speak about their organizations

- Outlined strategies for career fairs. Researching companies, preparing a strong elevator pitch, and having a well written resume.

- Broke down how to build a clean, one-page resume using Jake's Template and action verbs. Ended with an overview of the STAR method and how to prepare for technical interviews.

Learning Objectives

By The End Of This Lecture You Will Be Able To

- **Identify** shell types and when each is used
- **Read** an unfamiliar command and explain each part
- **Use** core commands to view, search, and retrieve files and data
- **Explain** how data flows through a command pipeline
- **Construct** simple regex patterns to filter data

Bash Review

Command Structure

```
command [flags] [arguments]
```

- **Command** – Action to perform. Always comes first. Example: ls, cd, mkdir
- **Flags** – Optional modifiers that change how the command behaves, starting with - or --. Example: -a, --all
- **Arguments** – The target the command acts on, like a file or directory. Depending on a command it might take arguments. Example: documents, new_folder

Try these Commands

```
echo “hello world”

pwd
```

What should be the output?

```
~$ “hello world”

~$ /your/working/directory
```

Different Shells

The Shell vs. the Terminal

Terminal — the window you type in; just a display

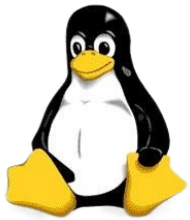
Shell — the program running inside it; interprets your commands

The terminal is the interface, the shell is the engine.

sh

the original Unix shell

Found on nearly every system



UNIX

(Bash before it was bash)

- Before it was “Bourne Again”, it had No command history, advanced auto-completion, and expanded scripting capabilities.

zsh

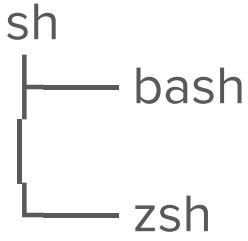
Z Shell

- Default on macOS since 2019
- More interactive features than bash
  - better autocomplete
  - smarter suggestions
  - highly customizable
- Compatible with bash syntax for the most part

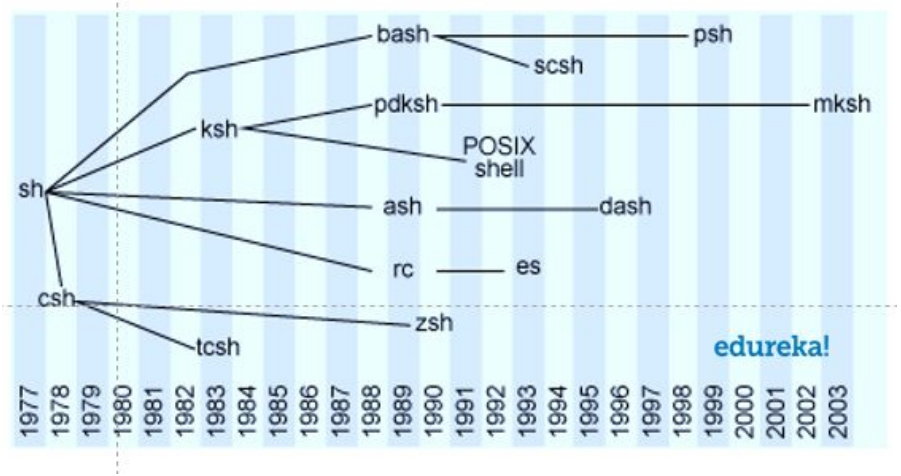


sh → bash → zsh is a rough line of evolution in everyday use

Shell Evolution



More Expansive graphic



POSIX is what standardized the interface that higher-level languages use to interact with the operating system.

Bash Shortcuts

Bash Shortcuts

- **`Ctrl + A`** - Jump to beginning of line
- **`Ctrl + Z`** - Jump to end of line
- **`<tab>`** - Autocomplete
- **`Ctrl + R`** - Search command history
- **`Up & Down`** - Command History
    - zsh searches your history to autocomplete
- **`Ctrl + C`** - Terminate command
- **`Ctrl + Z`** - Suspend command
- **`Ctrl + L`** - Clear screen



Thinking Exercise

Actions on the Computer

Consider these tasks...

1. Finding every file on your computer with “hello” in the name
2. Preview a huge file without opening it
3. Downloading a webpage’s content to search through later
4. Find every phone number in a giant text file
5. Seeing 5 most recent error messages in a 10000-line log file

Takes a lot of clicks... and time

1. Open file explorer, click to the overarching pc folder, click the search bar, type hello
2. Opening a large file with a file viewer takes a lot of time
3. ...you get the idea

Where bash comes in

All of the tasks listed can be accomplished in one line of bash!

One action to open your terminal window, and a second to just type the command



Be on the lookout!!!

The bash commands that complete the 5 tasks listed earlier will be introduced throughout this lecture. We will revisit these at the end of the lecture, so be on the lookout for them!!!

New commands

Review of Command Structure

```
$ command [flags] [arguments]
```

- `$` — the prompt; shows you're in a shell
- `command` — the command (program*) you want to run
- `flags` — optional switches that modify behavior, start with `-`
- `arguments` — what the command acts on (a file, a word, a path)

```
$ ls -a dir1/ dir2/
```

- command → `ls`
- flag → `-a` (include hidden . files)
- arguments → `dir1/` (first directory), `dir2/` (second directory)

*A COMMAND IS A PROGRAM (code that runs when you call it in the terminal)

Input Output Commands

```
echo  cat  head  tail

find  grep  egrep  locate   which   whereis

curl  wget
```

echo

```
$ echo “hello world”
```

Output: hello world

FLAGS

-n – omit the trailing newline

-e – enable escape sequences

Arguments

string you would like to display

Description

Prints text or the value of a variable to stdout. Heavily used in shell scripts.

Viewing File Contents

cat

```
$ cat file.txt
```

Output: contents of file.txt

FLAGS

-n – show hidden characters like tabs and line endings

Arguments

One of more file paths

Description

Concatenates (combines file content) and prints file content to stdout

head

```
$ head -n 20 huge_file.txt
```

Output: first 20 lines of huge_file.txt

FLAGS

-n N – show first N **lines** (default 10)

-c N – show first N **bytes**

Arguments

File path

Description

Outputs the beginning of a file

tail

```
$ tail -n 5 file.txt
```

Output: last 5 lines of file.txt

FLAGS

-n N – show last N **lines** (default 10)

-c N – show last N **bytes**

-f – follow: keep file open and stream new lines in real time

Arguments

File path

Description

Outputs the end of a file. **-f** is especially useful for monitoring live logs

Identifying Files and Directories of Interest

find

```
$ find . -name "file.txt"
```

Output: Relative paths of every match it finds recursively starting from the current directory

FLAGS

-name filename – match by filename (case sensitive)
-iname filename – match by filename (case **in**sensitive)
-type f/d – match by type (files or directories)
-size +10M – match by size (greater than 10MB)
-mtime -<time> – modified within <time>
-exec – run a command on each match

Arguments

Starting directory

Description

Recursively searches for files/directories matching the given criteria. Minimum working is `find .`

which

```
$ which ls
```

Output: displays the full path of where the program for the command ls is located

FLAGS

-a – show all matching paths in PATH, not just the first

Arguments

Executable name to search for

Description

Shows the full path of an executable. Useful for confirming which version of a program will run, or whether it's installed at all.

grep

```
$ grep "hello" file.txt
```

Output: Every line containing “hello” in file `file.txt`

FLAGS

-r – recursive
-i – pattern matching is case insensitive
-n – show line numbers
-l – only show filenames
-v – invert (show lines that don’t match)
-c – count matches
-E – Extended Regular Expressions (more on that later)

Arguments

Pattern, then file or directory to search

Description

Searches inside files for lines matching a pattern. One of the most-used commands for navigating codebases and logs

Retrieving Data

curl

```
$ curl https://example.com/
```

Output: a progress bar and the HTML content of the Example Domain page

FLAGS

-s – hide the progress bar (silent mode)

-o – save output to file

Arguments

URL

Description

Transfers data to or from a URL. Supports many protocols. Widely used for testing APIs and fetching web content.

curl activity - try the commands below

```
1  curl https://ifconfig.me
2  curl https://wttr.in/west-lafayette
3  curl dict://dict.org/d:bash
```

What did each return?



Answers

1

Your public IP Address

2

Weather in the big WL!!!

```
Weather report: west-lafayette

      \   /     Sunny
       .-.      +51(48) °F
    – (   ) –   ↑ 9 mph
       `-’      9 mi
      /   \     0.0 in

                                                       Tue 12
┌─────────────────────────────┬─────────────────────────────┐
│            Morning          │             Noon            │
├─────────────────────────────┼─────────────────────────────┤
│      \   /     Sunny        │      \   /     Sunny        │
│       .-.      51 °F        │       .-.      71 °F        │
│    – (   ) –   ↑ 9-14 mph   │    – (   ) –   ↑ 12-15 mph  │
│       `-’      9 mi         │       `-’      6 mi         │
│      /   \     0.0 in | 0%  │      /   \     0.0 in | 0%  │
└─────────────────────────────┴─────────────────────────────┘

                                                       Wed 13
┌─────────────────────────────┬─────────────────────────────┐
│            Morning          │             Noon            │
├─────────────────────────────┼─────────────────────────────┤
│ _`/"".-.    Patchy rain ne… │ _`/"".-.    Patchy rain ne… │
│  ,\_(   ).  +50(46) °F      │  ,\_(   ).  +51(48) °F      │
│   /(___(__) ↘ 16-20 mph     │   /(___(__) ↘ 17-21 mph     │
│     ‘ ‘ ‘ ‘ 6 mi            │     ‘ ‘ ‘ ‘ 6 mi            │
│    ‘ ‘ ‘ ‘  0.0 in | 84%   │    ‘ ‘ ‘ ‘  0.0 in | 60%   │
└─────────────────────────────┴─────────────────────────────┘

                                                       Thu 14
┌─────────────────────────────┬─────────────────────────────┐
│            Morning          │             Noon            │
├─────────────────────────────┼─────────────────────────────┤
│      \   /     Sunny        │      \   /     Sunny        │
│       .-.      +48(44) °F   │       .-.      +55(51) °F   │
│    – (   ) –   ↘ 10-12 mph  │    – (   ) –   ↓ 11-13 mph  │
│       `-’      6 mi         │       `-’      6 mi         │
│      /   \     0.0 in | 0%  │      /   \     0.0 in | 0%  │
└─────────────────────────────┴─────────────────────────────┘
Location: West Lafayette, Tippecanoe County, Indiana, USA [40.42

Follow @igor_chubin for wttr.in updates
```

3

Dictionary definition of bash

```
220 dict.dict.org dictd 1.12.1/rf on Linux 4.19.0-10-amd64 <auth.mime> <71323733
250 ok
150 3 definitions retrieved
151 "Bash" gcide "The Collaborative International Dictionary of English v.0.48"
Bash \Bash\, v. t. [imp. & p. p. {Bashed}; p. pr. & vb. n.
   {Bashing}.] [Perh. of imitative origin; or cf. Dan. baske to
   strike, bask a blow, Sw. basa to beat, bas a beating.]
   To strike heavily; to beat; to crush. [Prov. Eng. & Scot.]
   --Hall Caine.
   [1913 Webster]

         Bash her open with a rock.              --Kipling.
   [Webster 1913 Suppl.]
.
151 "Bash" gcide "The Collaborative International Dictionary of English v.0.48"
Bash \Bash\, n.
   1. a forceful blow, especially one that does damage to its
      target.
      [PJC]

   2. a elaborate or lively social gathering or party.
      [PJC]
.
151 "Bash" gcide "The Collaborative International Dictionary of English v.0.48"
Bash \Bash\, v. t. & i. [OE. baschen, baissen. See {Abash}.]
   To abash; to disconcert or be disconcerted or put out of
   countenance. [Obs.]
   [1913 Webster]

         His countenance was bold and bashed not. --Spenser.
   [1913 Webster]
.
250 ok [d/m/c = 3/0/19; 0.000r 0.000u 0.000s]
221 bye [d/m/c = 0/0/0; 0.000r 0.000u 0.000s]
```

wget

```
$ wget https://example.com/
```

Output: text indicating command progress

FLAGS

-q – quiet mode

-O filename – save to specific filename

-r – recursive download

Arguments

URL

Description

Downloads files from the web. Better than curl for large downloads thanks to automatic retry and resume support.

man

```
$ man man
```

Output: manual page for the command

FLAGS

-k – keyword search

-f – short description instead of full manual page

-a – displays all available man pages instead of just one

Arguments

command

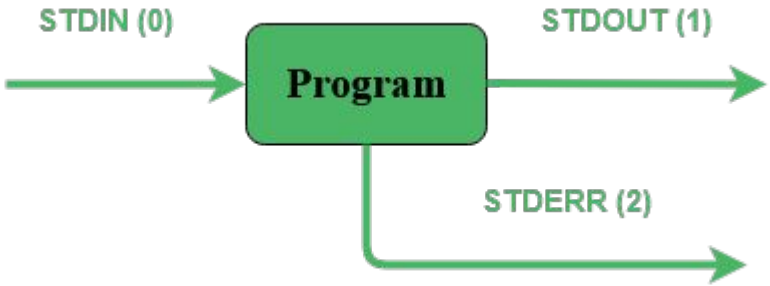
Description

Displays system reference manual pages for all default bash commands

I/O Redirection

stdin, stdout, stderr

0. **stdin**
   a. Standard input
   b. Default is your keyboard input
1. **stdout**
   c. Standard output
   d. Default is your terminal window → output gets printed to screen
2. **stderr**
   a. Standard error
   b. Error messages and diagnostics
   c. By default it is printed to the terminal as a separate stream



Taking Input from a File

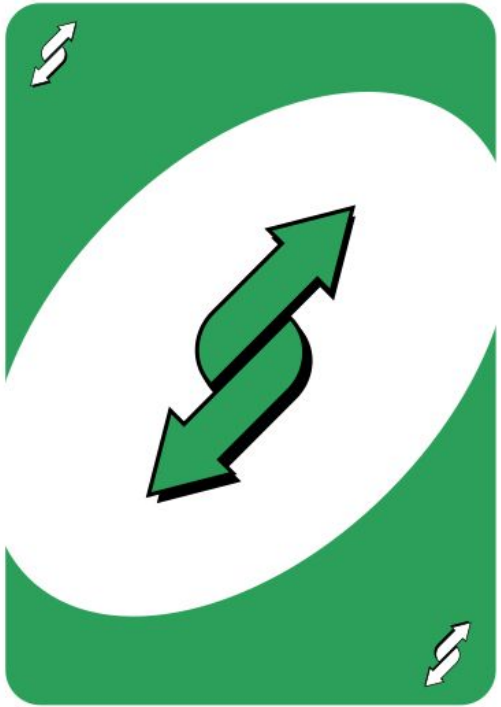
command < file

Redirect standard input from a file

Example:

```
ryansierra — -zsh — 80×24

(base) ryansierra@Ryans-MacBook-Pro-7 ~ % cat server.log
[14:01:59] Boot successful
[14:02:01] System error: connection timed out
[14:02:05] Retry successful
(base) ryansierra@Ryans-MacBook-Pro-7 ~ % grep "error" < server.log
[14:02:01] System error: connection timed out
(base) ryansierra@Ryans-MacBook-Pro-7 ~ %
```



Pushing Output to a File

command > file

- Redirect standard output to a file

command 2> file

- Redirect standard error to a file

command &>

- Redirect standard output and standard error to a file

command >> file

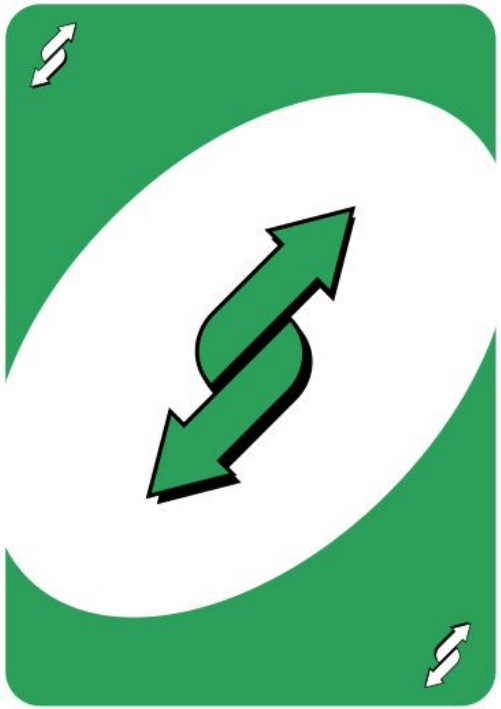
- Append standard output to a file

command 2>> file

- Append standard error to a file

command &>> file

- Append standard output and error to a file



Basic IO redirection example

```
$ curl https://example.com > page.txt
```

What does it do?

Reads a webpage, saves it to file page.txt

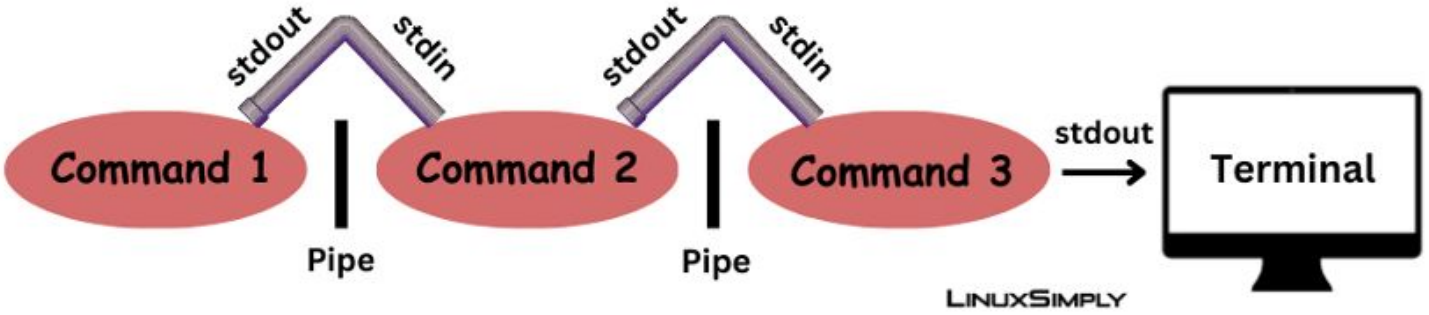
Input	Output	Commands
stdin (keyboard input) –in this case it's file.txt being typed by the user	page.txt	curl

Piping

Piping

Allows you to link multiple commands together!!

Output of one command becomes the input of another



Piping Syntax

command1 | command2 | command3 | ... | command

The output of the last command gets printed to stdout



Basic piping example

```
$ grep "error" log.txt | tail -n 5
```

What does it do?

Reads a log file, filter for lines containing “error”, and prints the most recent 5

Input	Output	Commands
stdin (keyboard input) –in this case it’s file.txt being typed by the user	stdout (terminal window)	grep, tail

Basic Piping Example Cont.

```
$ grep "error" log.txt | tail -n 5
```

log.txt content

```
Hello
This is an error #1
Eek error
More errors
Gasp error
errorerrorerror

penguin
Haha!
error
```

output

```
Eek error
More errors
Gasp error
errorerrorerror
error
```

Regular Expressions

Regular Expressions

a sequence of characters that define a search pattern

- Many bash commands allow you to use a regular expression as a parameter
  - e.g grep!
- Very helpful for writing shell scripts ➜ future lecture

What does a Regex Look Like?

A regex is just a pattern you write to describe what you're looking for in text. Think of it like a "smart search."

Example: `cat`

This matches any text containing the word `"cat"` — like `"cat"`, `"catch"`, or `"concatenate"`

Regex Symbols (more complex searches)

. (dot) — any single character | Example: c.t → matches “cat”, “cut”, “cot”

***** (star) — zero or more of the previous char | Example: ca*t → matches “ct”, “cat”, “caat”

+ (plus, ERE only) — one or more of the previous char | Example: ca+t → matches “cat”, “caat” (not “ct”)

ERE is a modernized standard for regular expressions that lets you use advanced search operators directly without needing to use backslash escape characters.

More Regex Symbols

[ ] (brackets) — any one character in the set | Example: gr[ae]y → matches “gray” or “grey”

^ and **$** (anchors) — start / end of line | Example: ^Hello matches lines starting with “Hello”; world$ matches lines ending in “world”

{ } (curly braces, ERE) — specific count of the previous char | Example: ca{2}t → matches “caat” (not “cat” or “caaat”)

Regex example

Example: `^[A-Z][a-z]+$`

Breaking it down:

- `^` — must start at the beginning of the line
- `[A-Z]` — first character is an uppercase letter
- `[a-z]+` — followed by one or more lowercase letters
- `$` — must end here

Matches: "Hello", "Alice", "John"

Does not match: "hello", "HELLO", "John123"

This is a pattern for a properly capitalized single word

Extended Regular Expressions

A more expressive standard that removes the need for backslash escaping on most metacharacters.

Key characteristics:

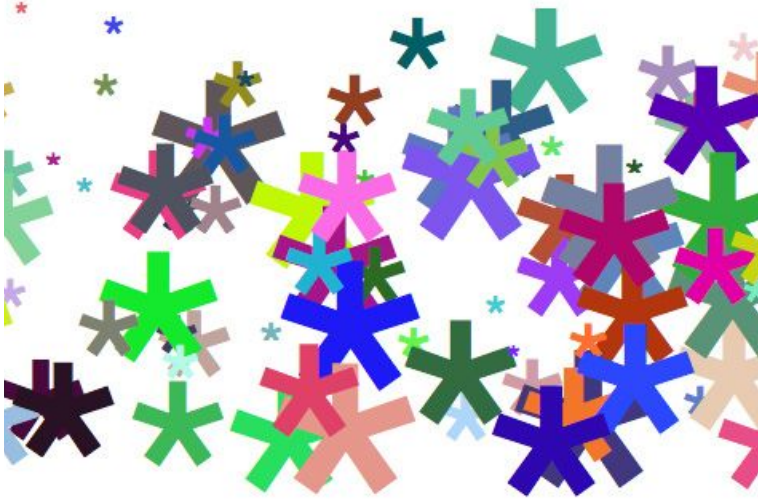
- Most matching characters must be escaped with a backslash to be treated as special
  - +, ?, and | are literal characters unless escaped: \+, \?, \|
- Cleaner, more readable syntax
- Supported by: `grep -E`, `egrep`, `awk`, `sed -E`

Example: `grep -E 'colou?r' file.txt` — matches "color" or "colour"

Regex Resources

The best way to learn about regular expressions is to make them yourself!

https://regexone.com/
https://regexr.com/
https://regex101.com/



Further reading about the depth of regex

https://www.regular-expressions.info/

Regex Example with `grep`

```
$ grep -E "[0-9]{3}-[0-9]{3}-[0-9]{4}" contacts.txt
```

What does it do?

Finds every phone number in the file `contacts.txt`

Input	Output	Commands
stdin (keyboard input) –in this case it's . and "*.txt" being typed by the user	Stdout (terminal screen)	grep

Tying everything together

From our 5 examples...

What are each of the commands we constructed to accomplish these?

1. Finding every file on your computer with “hello” in the name
2. Preview the start of a huge file without opening it
3. Downloading a webpage’s content to search through later
4. Find every phone number in a giant text file
5. Seeing 5 most recent error messages in a 10000-line log file

(hint: scroll back through the slides)

Construct your own command

Task: “Get the homepage of the purdue cs website, find every line that mentions "Computer Science", get the first 5 results, and save that to a file.”

Step 1: Deconstruct the Task

These will be the commands you use

1. get the homepage of the purdue cs website
2. find every line that mentions "Computer Science"
3. save the first 5 results of that to a file.

Step 2: assign a command to each portion

These will be the commands you use

1. curl
2. grep
3. Head

Step 3: chain

Always piping in-between commands

Think about ends: do you need to take a input from a file? Do we need to save output to a file?

For this example, we need to save output to a file!!!

Try it on your own

Final

curl https://www.cs.purdue.edu/ | grep "innovative" | head -n 10 > results.txt

Lecture Review

Rapid Fire Questions

What's the difference between a shell and a terminal?
What's the regex symbol for matching one or more characters?

Main takeaway: bash is a **scripting language** treat it as a flow of information and a means to get information you want

Quiz Time!

References

References

1 https://www.edureka.co/blog/types-of-shells-in-linux/